PromptFootprint

Complete Technical Breakdown

How the extension identifies AI models, captures prompts and responses, estimates energy, water and carbon, optimizes prompts, stores and syncs data — down to the formulas, scoring weights, selectors and data structures in the source.

Item	Detail
Repository	sahilp15/prompt_footprint (directory Prompt-Footprint-main/)
Product	Chrome Manifest V3 extension + React dashboard + Cloudflare Worker + optional Supabase sync
Extension version	manifest 1.0 / package 1.1.0; storage schema 2; model catalog schema 1 (updated 2026-08-06)
Supported sites	chatgpt.com, chat.openai.com, claude.ai, gemini.google.com
Document basis	Direct reading of the source tree (≈98k lines incl. dictionary) plus live runs of the estimator under Node
Prepared	October 9, 2026

Contents

1. Executive Summary **4**

2. System Architecture **5**

2.1 Content-script load order . . . 6

2.2 Manifest permissions . . . 6

3. End-to-End Lifecycle of One Prompt **7**

4. Model Identification — Deep Dive **8**

4.1 Module map . . . 8

4.2 The separations that are never merged . . . 8

4.3 Provider adapters and their selectors . . . 9

4.4 Step 1 — Collecting candidates . . . 9

4.5 Step 2 — Scoring . . . 10

4.6 Step 3 — Picking the winner (priority rule) . . . 10

4.7 Step 4 — Canonicalization (catalog.js) . . . 10

4.8 Step 5 — Building the observation . . . 11

4.9 Reasoning / thinking detection . . . 12

4.10 Tool and mode chips . . . 12

4.11 Confidence (detection, not estimate) . . . 13

4.12 Worked scoring examples . . . 13

4.13 The normalized DetectedModel record . . . 13

4.14 Live change detection (detector.js) . . . 14

4.15 Per-message snapshots (snapshots.js) . . . 14

4.16 Models the catalog has never seen (discovery.js) . . . 15

4.17 Debug panel and known limitations . . . 15

5. Prompt and Response Capture **16**

5.1 Platform adapters (platforms.js) . . . 16

5.2 Submit hook . . . 16

5.3 Response polling . . . 16

6. Token Estimation **17**

6.1 Prompt-size indicator (promptSize.js) . . . 17

7. Environmental Estimator **18**

7.1 Evidence classes . . . 18

7.2 Verified anchors . . . 18

7.3 Model priors (ordinary short interaction: ~100 in / ~300 out, no tools) . . . 18

7.4 High-reasoning / agentic priors . . . 19
7.5 Assembly algorithm, step by step . . . 19
7.6 Carbon . . . 20
7.7 Water . . . 20
7.8 Worked examples (actual estimator output) . . . 20
7.9 Display precision . . . 21
7.10 Context carried with every estimate . . . 21
8. Prompt Savings Math 22
9. Legacy Model, Heatwave Overlay and Equivalents 23
9.1 Legacy per-token model (storage schema 1) . . . 23
9.2 Heatwave / cooling overlay . . . 23
9.3 Real-world equivalents . . . 23
10. Token Cutter — Local Prompt Optimizer 24
10.1 Vetoes . . . 24
10.2 Confidence and levels . . . 24
10.3 "Already concise" — seven conditions . . . 25
10.4 Memory . . . 25
10.5 Optional Gemini enhanced pass . . . 25
10.6 Quality evaluation . . . 25
11. In-Page Writing Assistant 26
11.1 Offline spell / writing checker . . . 26
12. AI Proxy and Service-Worker Rate Control 27
12.1 Cloudflare Worker (proxy/worker.js) . . . 27
12.2 Service-worker client . . . 27
13. Local Storage and Data Model 28
13.1 Per-query record . . . 28
14. Optional Accounts and Cross-Device Sync 29
14.1 Schema (supabase/migrations/0001_init.sql) . . . 29
15. Dashboard, Popup and Public Site 30
16. Security and Privacy Model 31
17. Testing and Quality 32
18. Observations From This Review 33
19. Appendix A — File Map 34
20. Appendix B — Sources Referenced by the Code 35

1. Executive Summary

PromptFootprint is a browser extension that sits on top of ChatGPT, Claude and Gemini and shows the estimated environmental cost — energy (Wh), water (mL) and carbon (gCO_2e) — of each prompt, before you send it and after the answer arrives. It also ships a local prompt optimizer (the **Token Cutter**) that removes filler and redundancy while proving it kept every requirement, and a dashboard of sessions, weekly trends, savings and awards.

The engineering is organized around one idea, stated repeatedly in the code: *be aggressive about finding reliable signals, and conservative about claiming what the product does not expose.* Concretely:

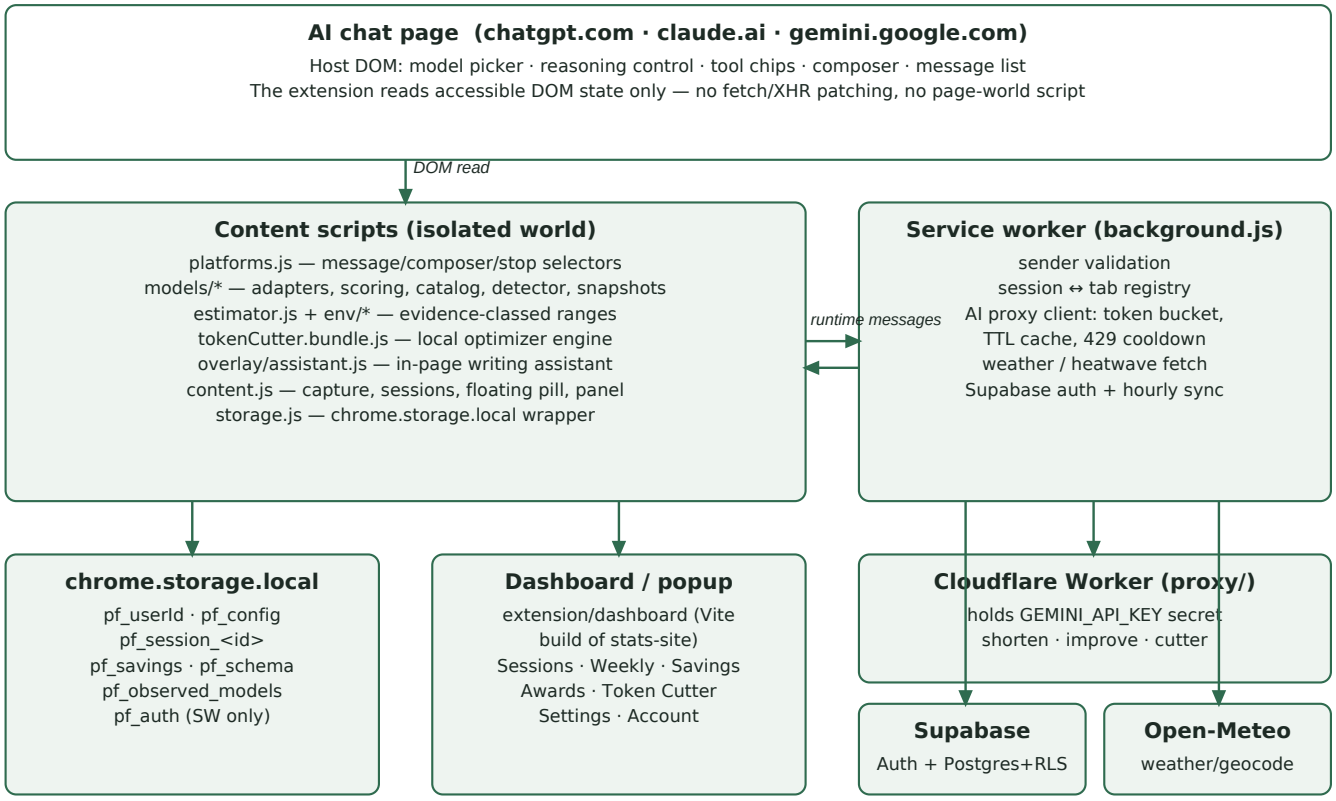
- **Model identification is read from accessible DOM state** (the same state a screen reader sees): the model picker, the checked row of an open menu, the reasoning/effort control and tool chips. Every candidate control is scored on independent semantic signals (+40 selected, +25 known token, +20 near composer, +15 ARIA terms, +10 visible; −30 hidden, −30 unselected menu row, −25 destructive/settings). The winner must clear a floor of 35, otherwise the answer is honestly "unknown".
- **Labels are canonicalized through a versioned data catalog** with a version guard: a label stating a version ("GPT-5.7 Sol") must agree with the catalog family ("gpt-5.6") or it is refused rather than silently mapped to last generation's model.
- **Modes are not models.** Auto/Instant/Thinking/Pro, Projects, Gems, custom GPTs and styles are recognized as what they are; Auto never resolves itself into a model name.
- **The model is frozen per message at send time** (snapshots), so switching models afterwards never rewrites history. Prompt text is never stored — only an FNV-1a hash and a token count.
- **Estimates are ranges with an evidence class** (MEASURED / REPORTED / MODELED / ENGINEERING_PRIOR), built from published anchors (Google's 0.24 Wh Gemini median, OpenAI's reported 0.34 Wh, Jegham et al. proxies, Oviedo et al. frontier distributions) and model-specific engineering priors.
- **Local-first:** everything works offline with no account. Optional Gemini rewriting goes through a Cloudflare Worker that holds the API key; optional Supabase sync uploads numbers only, under row-level security.

> **Reading guide.** Section 4 is the deep dive on **how models are identified**. Sections 6–9 explain the environmental math. Section 10 covers the Token Cutter. Sections 13–16 cover storage, sync, security and testing. Section 17 lists issues observed while preparing this document.

2. System Architecture

The repository contains six deployable or reference components. Only the extension is required; everything else is optional or legacy.

Component	Path	Technology	Role
Browser extension	manifest.json, extension/	Plain JS (no bundler) content scripts + MV3 service worker	Detection, capture, estimation, overlay, in-page assistant, local storage
Dashboard / stats site	stats-site/ → extension/dashboard/	React 19, Vite, Tailwind 4, Recharts, TypeScript engine	Options page inside the extension and a public marketing/app site
Token Cutter engine	stats-site/src/lib/tokenCutter/*.ts	TypeScript, compiled by esbuild to an IIFE	Prompt optimizer shared by dashboard and in-page assistant
AI proxy	proxy/worker.js	Cloudflare Worker	Holds the Gemini key; shorten / improve / cutter modes
Accounts & sync	supabase/	Supabase Auth + Postgres + RLS	Optional cross-device sync of numeric summaries
Legacy server	server/	Express + Sequelize + Postgres	Former backend, kept for reference; not used



2.1 Content-script load order

Because there is no bundler for content scripts, modules are plain IIFEs that publish a global (`PFModelCatalog`, `PFEstimator`, …) and also export via `module.exports` so the same files run under Node tests. The manifest loads them in dependency order:

```
storage → proxyConfig → platforms → constants → tokenEstimator → environmentalModel
→ env/sources → env/profiles → env/factors → env/copy → estimator
→ models/catalog → models/reasoning → models/observation → models/discovery
→ models/adapters/shared → adapters/openai → adapters/anthropic → adapters/google → adapters/index
→ models/detector → models/snapshots → models/present
→ writingFormat → uiHelpers → formatters → tokenCutter.bundle → composer → assistantState
→ overlay/assistant → content.js        (run_at: document_idle)
```

2.2 Manifest permissions

Permission / host	Why it is needed
storage	All persistence (chrome.storage.local / .session)
activeTab, alarms	Popup interaction; hourly pf_sync alarm
geolocation	Optional heatwave overlay location (user-chosen)
chatgpt.com, chat.openai.com, claude.ai, gemini.google.com	Content-script injection
*.workers.dev	Gemini proxy Worker
cpwbtulpufhqexrrzjzt.supabase.co	Optional accounts/sync
api.open-meteo.com, geocoding-api.open-meteo.com	Weather for the heatwave overlay

Extension pages run under a strict CSP: `script-src 'self' 'wasm-unsafe-eval'; object-src 'none'; frame-src 'none'`, with `connect-src` limited to the Worker, Supabase and Open-Meteo.

3. End-to-End Lifecycle of One Prompt

This is what happens from page load to a saved, estimated interaction:

```
1. Page load (document_idle)
   content.js → inject floating pill + modal immediately
              → PFStorage.getUserId() (anonymous UUID) and getConfig()
              → createSession; REGISTER_SESSION {sessionId} → SW maps tab ↔ session
              → startModelDetection(): PFModelDetector.create({adapter}).start()
                   first observation = discovery (no change event fired)
              → setupSubmitHook(), setupDraftWatch(), startAssistant(), MutationObserver fallback

2. User types
   'input' event → 400 ms debounce → recomputeDraft('typing')
     tokens = ceil(len/4) → PFEstimator.estimate({...observation, phase:'draft'})
   assistant: 600 ms pause → PFTokenCutter.analyzePrompt() → suggestions + savings range

3. User switches model / effort
   detector: mutation → 120 ms scan → fingerprint differs → 350 ms recalc
     → identity changed → generation++ → onModelChange(current, previous)
     → pill relabels, in-flight draft work cancelled, draft re-estimated, optimizer re-run

4. User presses Enter / clicks Send   (capture phase listeners, before the host clears the box)
   captureSubmittedPrompt(): read composer text, 3 s dedup window
     → takeSendSnapshot(): freeze observation + token count + pre-send estimate (+ FNV-1a hash)
     → pill = "Recording…" → startResponseWatch() polling every 500 ms

5. Response streams
   poll latest assistant element; text growing or Stop button / streaming flag present → keep waiting
   complete when not generating AND text stable ≥ 2000 ms (or platform completion signal)
   give up after 30 s with no text; force-finalize at 240 s

6. Finalize
   processQuery(): legacy per-token impact → replaced by completionEstimate(snapshot)
     → PFStorage.addQuery(session, {tokens, energy/water/CO2 central + band, boundaries,
        evidence, confidence, model ids, routing, reasoning, tools, lowerBound, versions})
     → session totals updated → pill "Saved" → modal refreshed
```

4. Model Identification — Deep Dive

Model identification lives in `extension/lib/models/`. Everything downstream depends on it: the environmental estimate is built from the model's prior and its reasoning tier (which can move energy by an order of magnitude), and the Token Cutter consults it too. The design premise, from `docs/MODEL_DETECTION.md`: *a detector that is confidently wrong is worse than one that says "not exposed".*

4.1 Module map

File	Responsibility
catalog.js	Versioned data table: aliases → canonical ids, families, tiers, modes, vendor thinking constraints; label normalization
observation.js	The observation record, candidate scoring, confidence formula, DOM helpers (hidden/selected/accessible name), scoped MutationObserver, toDetectedModel()
adapters/openai.js, anthropic.js, google.js	Per-provider selector lists, tool vocabularies, surface and conversation-key readers, product quirks (refine)
adapters/shared.js	Everything after "here are the candidates": pick, canonicalize, reasoning control, tool chips, constraints, confidence
adapters/index.js	Chooses the adapter for the current location (forLocation)
reasoning.js	Reasoning label vocabulary → estimator mode → reasoning class; label splitting ("Thinking · High")
detector.js	Live loop: throttled scan, debounced recalc, generation counter, change events
snapshots.js	Per-message frozen observations and estimates; FNV-1a prompt hash
discovery.js	Local registry of model labels the catalog does not know (pf_observed_models)
present.js	Shared wording for panel and popup; change explanations

4.2 The separations that are never merged

Pair	Meaning	Why keeping them apart matters
selectedLabel vs canonicalModelId	What the picker showed verbatim vs what we mapped it to (or null)	An unmapped label keeps its text and gets a null id; it is never rounded to the nearest flagship
selectedModel vs effectiveModel	The user's choice (may be "auto") vs the model that actually served a response	Under Auto the effective model is null unless the provider exposes it
verified vs estimateBasis	Did the product tell us which model is selected? vs do we have a calibrated footprint for it?	A model released this morning can be fully verified while its estimate is a provider-level fallback

4.3 Provider adapters and their selectors

An adapter is mostly data: ordered selector lists (current test-ids first, role/ARIA fallbacks after, so a class-name reshuffle does not break detection), a tool vocabulary, and a few quirks. If every selector misses, detection returns "unknown" and the page keeps working — the extension never blocks the app.

	ChatGPT (openai)	Claude (anthropic)	Gemini (google)
Hosts	chatgpt.com, chat.openai.com	claude.ai	gemini.google.com
Picker	[data-testid="model-switcher-dropdown-button"], [data-testid="model-switcher"], #model-switcher-button, button[aria-label*="model" i], …	[data-testid="model-selector-dropdown"], [data-testid="model-selector"], button[data-testid*="model" i], button[aria-label*="model" i], …	[data-test-id="bard-mode-menu-button"], [data-test-id="mode-switch-button"], button.gds-mode-switch-button, .mode-switcher button, …
Menu rows	[role=menu] [role=menuitemradio|menuitem], [role=listbox] [role=option], radix menu content	same ARIA roles	same ARIA roles + .mat-mdc-menu-panel .mat-mdc-menu-item
Reasoning control	data-testid / aria-label containing reasoning, thinking, effort	data-testid / aria-label containing effort, thinking	data-test-id / aria-label containing thinking, reasoning, deep think
Observed roots	#page-header, header, unified-composer form, #composer-background, main, menus	header, fieldset, main, chat-input-container, menus	input-container, rich-textarea, main, header, .input-area, menus
Surface	/g/ → custom-gpt, else chatgpt	always claude-web (Projects are not models)	/gem/ → gem, else gemini-web
Conversation key	openai:<id after /c/ or /g/>	anthropic:<id after /chat/ or /project/>	google:<id after /app/ or /gem/>
Quirks (refine)	custom-GPT name is never a model	Projects/styles excluded from candidates entirely	Deep Think sets reasoning deep-think; a Gem without exposed model → routing unknown

4.4 Step 1 — Collecting candidates

`collectCandidates()` gathers two kinds of element and turns each into a scored candidate with `candidateFrom(el, provider, opts)`:

- **Menu rows** (option rows). Marked `optionRow: true`, `nearComposer` if inside one of the adapter's trusted regions. Source is `selected-menu-item` if the row is marked current, else `aria`. Unselected rows are collected deliberately, so a hidden menu clone full of model names is penalized rather than ignored.
- **Picker buttons**. Source `picker-label`, always treated as near the composer.

For each element the candidate records: visible text (whitespace-squeezed, max 120 chars), accessible name (`aria-label` then `title`), the canonicalization result, the mode reading, hidden/disabled/selected state and an "unrelated" flag.

How "selected" and "hidden" are decided

- **Selected:** `aria-selected="true"`, `aria-checked="true"`, `data-state="checked"`, `aria-current="true"`, or a check icon inside the row (`[data-testid*="check" i]` / `[aria-label*="selected" i]`).

- **Hidden:** walking up to 30 ancestors — `hidden` attribute, `aria-hidden="true"`, `inert`, `data-state="closed"`, a `<template>`, or inline `display:none`/`visibility:hidden`. It reads semantics, not layout boxes, so it behaves identically under jsdom tests.
- **Unrelated:** text or ARIA matches `delete|remove|clear|discard|log out|sign out|settings|preferences|share|archive|report|upgrade|subscribe|billing`.
- **ARIA terms:** accessible name matches `model|gpt|chatgpt|claude|gemini|sonnet|opus|fable|mythos|haiku|sol|terra|luna|flash|pro|thinking|deep think|instant|auto`.

4.5 Step 2 — Scoring

Signal	Weight	Rationale
Marked selected (aria-selected / aria-checked / data-state=checked / aria-current / check icon)	+40	The product itself stating the current row
Contains a known model or mode token (canonicalize or readMode succeeds)	+25	Provider-specific, so a Gemini string cannot win on a ChatGPT page
Lives in the composer or top bar	+20	Where pickers actually live
Accessible name mentions model/mode terms	+15	Semantic hint independent of styling
Rendered and enabled	+10	
Hidden, collapsed menu, or template clone	−30	Real pages keep closed menus in the DOM
A row in an open menu that is not the current one	−30	Otherwise the first alternative would beat the picker
Destructive or settings control	−25	Has text, is not a model control

```
MIN_SCORE = 35          // a candidate must clear this to be trusted
pickBest(cands): highest score ≥ floor wins; ties keep DOM order (earlier control)
```

4.6 Step 3 — Picking the winner (priority rule)

```
named = candidates.filter(c => c.canon)          // controls that NAME A MODEL
best  = pickBest(named) || pickBest(candidates)   // model-naming first, if credible
if (!best):                                       // nothing cleared 35
    fallback = best-scoring visible, non-unrelated 'picker-label' candidate
    → keep its raw text as selectedLabel, unmappedLabel = true
else → unknown
```

Both halves are load-bearing. Without preferring model-naming candidates, opening the *effort* submenu lets its checked row ("Max": 40 selected + 25 token + 10 visible = 75) beat the model picker, and the model is lost. Without the "credible" requirement (must clear the floor), a *collapsed* menu full of model names would suppress the visible picker whenever it shows a mode like Auto.

4.7 Step 4 — Canonicalization (catalog.js)

Normalization

`normalizeLabel()` applies Unicode NFKC folding, lowercases, unifies the hyphen/dash/minus family to "-", straightens smart quotes, drops ™ ® © ℠ and dropdown carets (▾ ▼ ⌄ ↓), turns other decoration into spaces and collapses whitespace. It deliberately leaves **digits, dots and tier words** untouched — those are the difference between Sonnet 5 and Opus 5, or Gemini 3.1 Pro and 3.6 Flash.

Matching

- Exact alias lookup on the normalized label; otherwise aliases are tried **longest-first** (so "gemini 3.1 deep think" beats "deep think") with whole-token matching: `(?:^|[^a-z0-9])alias(?![a-z0-9])` — "highlight" never matches "high".
- **Version guard:** if the label contains a version number (`/\d+(?:\.\d+)?/`) it must appear in the resolved model's family string, or the result is `null`. "GPT-5.7 Sol" therefore does *not* resolve to gpt-5.6-sol; it is preserved as an unmapped label and estimated at the provider level.
- The result carries `canonicalModel, matchedAlias, normalizedLabel, family, tier, displayLabel, sourceIds` so the UI can explain *why* a label resolved.

Catalog contents (schema 1, updated 2026-08-06)

Provider	Canonical id	Aliases (normalized)	Family / tier	Notes
openai	gpt-5.6-sol	gpt-5.6, gpt 5.6, gpt-5.6 sol, gpt5.6 sol, sol	gpt-5.6 / flagship	1.05M ctx, 128k out, effort none…max
openai	gpt-5.6-terra	gpt-5.6 terra, terra	gpt-5.6 / balanced	
openai	gpt-5.6-luna	gpt-5.6 luna, luna	gpt-5.6 / efficient	
anthropic	claude-sonnet-5	sonnet 5, claude sonnet 5	claude-5 / balanced	thinking supported-default
anthropic	claude-opus-5	opus 5, claude opus 5	claude-5 / advanced	thinking on by default; locked at xhigh/max
anthropic	claude-fable-5	fable 5, claude fable 5	claude-5 / frontier	adaptive thinking always on
anthropic	claude-mythos-5	mythos 5, claude mythos 5	claude-5 / restricted-frontier	adaptive thinking always on
google	gemini-3.1-pro	gemini 3.1 pro, 3.1 pro, gemini pro	gemini-3.1 / flagship	1M ctx, 64k out
google	gemini-3.1-deep-think	deep think, gemini 3.1 deep think	gemini-3.1 / reasoning-mode	built on 3.1 Pro
google	gemini-3.6-flash	gemini 3.6 flash, 3.6 flash	gemini-3.6 / efficient	higher version ≠ more compute

Modes (labels that describe how, not which)

Provider	Mode label	routing	reasoning
openai	auto	auto	unknown
openai	instant / thinking mini / thinking / pro	unknown	none / low / high / pro
anthropic	adaptive thinking / extended thinking	fixed	adaptive / high
google	fast / thinking / deep think	fixed	none / high / deep-think
any	effort words: none, minimal, off, low, medium, standard, high, xhigh, extra high, max, maximum, adaptive	fixed	mapped level

4.8 Step 5 — Building the observation

```
obs.selectedLabel = best.label; obs.source = best.source
if best.canon → canonicalModel, family, tier
if best.mode  → routing, reasoningMode (unless 'unknown')
if routing == 'auto' and a model was named in the same control:
       effectiveModel = canonicalModel; canonicalModel = null      // intent stays "auto"
elif canonicalModel and routing == 'unknown': routing = 'fixed'
if dedicated reasoning control found → reasoningMode/Label/Source override the label-inferred value
if provider response metadata given  → effectiveModel, source = 'response-metadata'
obs = adapter.refine(obs);  obs = applyModelConstraints(obs);  obs.confidence = …
```

4.9 Reasoning / thinking detection

Reasoning is a separate control, read separately, and is the second-largest input to the estimate. `readReasoningControl()` runs two passes: (1) a dedicated visible reasoning control; (2) the selected row of an open menu whose label describes reasoning but does **not** canonicalize to a model (this is what stops "GPT-5.6 Sol" being read as an effort level). Null means "not exposed" — a true statement — rather than an invented default.

Three vocabularies are kept and never merged:

Vocabulary	Values	Used for
Raw label	Exactly what the product showed, e.g. "Extended thinking", "Thinking · High"	Display only; never interpreted
Estimator mode	none · low · medium · high · xhigh · max · adaptive · pro · deep-think	Wired into the energy priors
Reasoning class	minimal · standard · adaptive · high · maximum · pro	Normalized bucket for UI and comparisons

Label → mode matching is longest-first over a vocabulary of ~45 phrases, e.g. instant/fast/off → none; light/thinking mini → low; standard/balanced → medium; thinking/extended thinking/think harder → high; x-high/extra high/ultra think → xhigh; max thinking → max; pro mode → pro; deep think/deep research → deep-think; auto/adaptive → adaptive. The class → mode round trip is lossy *in the safe direction*: maximum maps back to max (not xhigh) so it can never reduce assumed compute. `splitLabel()` splits combined labels on · | , / – — into a mode half and an effort half.

Vendor constraints that outrank any control

```
applyModelConstraints(obs):
  thinking == 'always-on-adaptive' (Fable 5, Mythos 5):
      reasoning none/unknown/missing → 'adaptive', reasoningLockedBy = 'model'
  thinking == 'on-by-default' (Opus 5):
      unknown/missing → 'adaptive';  'none' → 'adaptive', lockedBy = 'model-default'
      reasoning in thinkingLockedAt [xhigh, max] → lockedBy = 'model'
```

So a control reading "Off" next to Fable 5 does not produce a no-reasoning estimate.

4.10 Tool and mode chips

`readToolChips()` reads controls that are switched *on* (aria-pressed / aria-checked = true, data-state on/active, data-active) and maps their text through the adapter vocabulary, longest phrase first and one match per control (so "Deep Research" is not also counted as "search").

Provider	Visible wording → canonical tool id
ChatGPT	deep research→deep-research; search / search the web→web-search; browse→browsing; canvas; create image/image→image; code interpreter/data analysis/python→code-execution; computer use; agent; codex; study

Provider	Visible wording → canonical tool id
Claude	research→deep-research; web search/search→web-search; artifacts; analysis/code execution→code-execution; computer use; drive/connectors→connected-apps
Gemini	deep research/research→deep-research; canvas; create images/image→image; video; audio; google search/search→web-search; apps→connected-apps

4.11 Confidence (detection, not estimate)

```
c = clamp((score − 10) / 80, 0, 1)
if routing == 'auto' and no effectiveModel:  c = min(c, 0.50)
if no canonicalModel:                        c = min(c, 0.35)
if source == 'provider-default':             c = min(c, 0.30)
round to 2 decimals
```

Verified sources are exactly `selected-menu-item`, `picker-label` and `response-metadata`, and only when a label exists.

4.12 Worked scoring examples

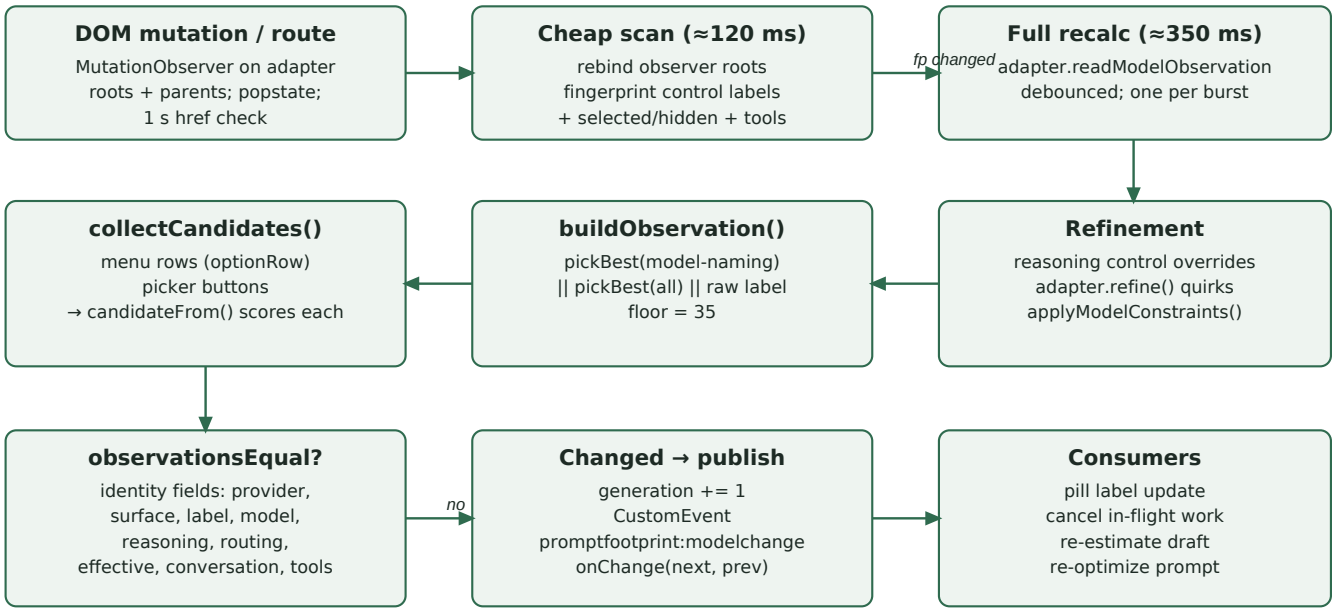
Scenario	Candidates and scores	Result
Claude, menu closed, picker shows "Opus 5"	picker-label "Opus 5": token 25 + near 20 + visible 10 (+15 if aria-label mentions model) = 55–70	canonical claude-opus-5; reasoning forced to adaptive by vendor rule; confidence 0.56–0.75; verified
ChatGPT, model menu open, "GPT-5.6 Sol" row checked	checked row: 40 + 25 + 20 + 10 = 95. Other rows: 25 + 20 + 10 − 30 = 25 (below floor). Picker: 55	selected-menu-item wins; confidence 1.0; verified
ChatGPT, effort submenu open, "Max" checked, picker shows "GPT-5.6 Sol"	"Max" row 75 (no canon); picker 55 (canon)	model-naming preference picks the picker (55 ≥ 35); reasoning control supplies max
ChatGPT picker shows "Auto"	picker 55 via mode token, no canon	routing auto, canonical null, confidence ≤ 0.35; estimate uses generic distribution, lower bound
Picker shows "GPT-7.2 Nimbus"	version guard rejects any alias; no token → 30–45	label preserved verbatim, unmapped; discovery records it; estimateBasis provider-fallback
Hidden closed menu containing "Fable 5"	row: token 25 + near 20 − hidden 30 − unselected 30 ≈ −15 (0 with ARIA terms)	ignored; never suppresses the visible picker

4.13 The normalized DetectedModel record

`toDetectedModel(observation)` projects every observation onto one cross-provider shape:

```
provider, product (chatgpt|claude|gemini), surface
selectedLabel            // exactly what the product rendered
canonicalModelId         // or null
family, variant, tier
selectedMode ('auto'|'fixed'), routing, effectiveModel
reasoningModeLabel, reasoningEffortLabel, reasoningMode, reasoningClass, reasoningLockedBy
tools[]
detectionSource, verified, unmapped, confidence
estimateBasis ('model' | 'provider-fallback')
conversationKey, rawEvidence[] (top-4 "source:label(score)"), detectedAt, generation
```

4.14 Live change detection (detector.js)



- **No DOM polling.** A MutationObserver watches only the adapter's roots (childList, subtree, characterData, and the attributes aria-label/checked/selected/current/expanded, data-state, data-testid, title, class). Each root's *parent* is watched shallowly too, because React *replaces* the header/composer subtree on navigation and a mutation of a detached node is heard by nobody.
- **Cheap scan (throttled 120 ms):** re-resolve roots and rebind if they changed (disconnect first, so observers never stack); compare `location.href`; compute a fingerprint of up to 40 model controls as `selected+hidden:text|aria` plus active tools. Only a changed fingerprint or route escalates.
- **Full recalc (debounced 350 ms):** build a full observation; if `observationsEqual` on the identity fields (provider, surface, selectedLabel, canonicalModel, reasoningMode, routing, effectiveModel, conversationKey, tools) only the timestamp is refreshed.
- **On change:** `generation += 1`, dispatch `promptfootprint:modelchange` {previous, current}, call onChange. The first read is a discovery, not a change, so no event fires on page load.
- **Routing:** popstate and hashchange listeners plus a 1 s interval that reads one string (href), because pushState fires no event and the extension refuses to inject page-world script to hear it.
- **Generation as cancellation:** any async result (estimate, optimizer pass) stamped with an old generation describes a model the user has left and is dropped (`isCurrentGeneration(gen)`).

4.15 Per-message snapshots (snapshots.js)

At send time the observation, the input token count and the pre-send estimate are frozen with a JSON clone (dropping DOM references). Later picker changes cannot touch them. The only amendment allowed is `refineEffectiveModel(id, info)`, which requires provider metadata naming the model that served that specific response. The store is an in-memory ring of 200 entries. Completion attaches `estimateAfterResponse` beside — never over — the pre-send estimate.

```
hashPrompt(text):            // FNV-1a 32-bit, identity check only — never the prompt itself
  h = 0x811c9dc5
  for each UTF-16 code unit c:  h ^= c;  h = imul(h, 0x01000193) >>> 0
  return "pf1:" + hex8(h) + ":" + text.length
```

4.16 Models the catalog has never seen (discovery.js)

- Capture the exact visible label and keep showing it — there is no "Probably GPT-5.6".
- Record it locally in `pf_observed_models` (ring of 50; provider, label, normalized, first/last seen, count). Never transmitted.
- Strict admission: ≤ 80 chars, not already canonical, not configuration-shaped (`project|gem|style|custom|assistant|agent|folder|workspace|untitled|new chat`), and must contain a digit or a family word (gpt, claude, gemini, sonnet, opus, haiku, fable, mythos, sol, terra, luna, flash, nano, mini, turbo, ultra, pro).
- Auto is a mode, not an unknown model, and is never recorded.
- The estimate falls back to the provider-neutral distribution and is flagged `estimateBasis: 'provider-fallback'`.

4.17 Debug panel and known limitations

Behind *Assistant settings → Model-detection debug panel* (off by default) the panel shows provider, product, detected and canonical model, raw and classified reasoning, detection source, verified flag, estimate basis, last change time and the **losing candidates with their scores**, so a detection bug is diagnosable from a screenshot.

- ChatGPT Auto routing is not observable; it reads as Auto and never resolves into a model name.
- A custom GPT's or Gem's backend may migrate; the surface is labelled honestly.
- Reasoning is only detected where the product exposes a control; otherwise "not exposed".
- The catalog is a snapshot: new models are named correctly but estimated at provider level until added.
- Selectors are inherently brittle; the layered fallbacks and scoring reduce, but do not remove, redesign risk.

5. Prompt and Response Capture

5.1 Platform adapters (platforms.js)

Separate from model adapters, these describe each site's message DOM: `id, name, hostMatches, rootSelector, messageSelector, inputSelector, stopSelector, sendSelector, getRole, getMessageId, getLatestAssistant, extractText, generatingSignal, isComplete`.

Platform	Messages	Generating signal	Completion signal
ChatGPT	[data-message-author-role] with data-message-id	Stop button (data-testid="stop-button" or aria-label Stop), .result-streaming	copy-turn action button present on the latest turn
Claude	user/assistant selectors; assigns data-pf-id when no native id	Stop button; [data-is-streaming="true"] unless the latest turn is complete	retry buttons count ≥ user turns
Gemini	user-query / model-response elements (custom tags or data-test-id)	Stop button ([data-test-id="stop-button"]), loading indicator	message-actions or copy button on the latest turn

`extractText()` clones the node and strips script, style, button and svg before reading textContent.

5.2 Submit hook

Two capture-phase listeners read the composer the instant the user sends, before the host clears it: Enter (not Shift+Enter, not during IME composition) inside the composer, and a click on the send button. PF`Composer.findComposer` accepts either the adapter selector or a heuristically detected composer, so a stale selector does not silently stop tracking. A 3 s dedup window prevents Enter+click double counting; a MutationObserver remains as a fallback for programmatic submits.

5.3 Response polling

Streaming replaces DOM nodes rather than mutating text, so a characterData debounce misses most of a stream. Capture therefore polls:

Constant	Value	Meaning
RESPONSE_POLL_MS	500 ms	Sampling interval of the latest assistant text
SETTLE_DELAY_MS	2000 ms	Text must be stable this long (and not generating) to finalize
SUBMIT_DEDUP_MS	3000 ms	Ignore duplicate triggers for the same turn
MAX_NO_RESPONSE_MS	30 s	Give up if no assistant text ever appears
HARD_CAP_MS	240 s	Force-finalize a response that never settles

While a generating signal is present the stability clock is reset each tick, so long "thinking" phases with momentarily stable text never finalize early. Duplicate finalization is guarded by comparing with the last finalized text and tracking processed message ids. A 0-token interaction is never persisted.

6. Token Estimation

```
estimateTokens(text) = max(1, ceil(trim(text).length / 4))     // 0 for empty input
```

The extension never sees provider token counts. cl100k_base averages ≈4 characters per token (code ≈3.5, prose ≈4.5), so 4 is a midpoint. Expected error is about ±15%, worse for non-English and heavily formatted text, and counts are not comparable across tokenizer generations (Claude Sonnet 5's tokenizer can produce ~30% more tokens than Sonnet 4.6 for the same text). The Token Cutter's `tokens.ts` mirrors this exactly; `stats-site/test/parity.test.ts` fails if the two implementations ever disagree.

6.1 Prompt-size indicator (promptSize.js)

Compares the current prompt against the user's *own* average. Below 15 tokens nothing is said. With fewer than 5 past prompts, neutral bands apply (≤60 short, ≤250 typical, otherwise long — "This is a long prompt, which may use more energy"). With history, a prompt ≥ 2× the personal average is flagged "much larger than your average"; otherwise it is classified short (≤0.6×) or typical with no nag.

7. Environmental Estimator

`extension/lib/estimator.js` turns (provider, selected/effective model, routing, reasoning, tools, tokens, phase) into a **range** with its evidence attached. Three rules: never a bare scalar; never a borrowed identity (a product median is not a model measurement, an unknown model never becomes a flagship); never a merged boundary (carbon and water carry their accounting scope).

```
interaction = fixed serving + input prefill + visible output decode
            + hidden reasoning + model/tool routing overhead
task        = Σ observed interactions   (marked a lower bound when calls are hidden)
```

7.1 Evidence classes

Class	Meaning
MEASURED	Production instrumentation with a disclosed methodology (only one profile: Gemini Apps product median)
REPORTED	Provider figure without enough method to reproduce it
MODELED	Independent estimate from hardware/latency/throughput/token assumptions
ENGINEERING_PRIOR	PromptFootprint assumption for a model nobody has measured

7.2 Verified anchors

Anchor	Energy	Carbon	Water	Class
Gemini Apps median text prompt, May 2025 [S1]	0.24 Wh	0.03 gCO2e	0.26 mL	MEASURED (product median)
Average ChatGPT query, June 2025 [S2][S3]	≈0.34 Wh	n/d	≈0.322 mL	REPORTED
GPT-4o short / medium / long, Azure proxy [S5]	0.423 / 1.215 / 2.875 Wh	proxy	proxy	MODELED
Claude 3.7 Sonnet short / medium / long, AWS proxy [S5]	0.950 / 2.989 / 5.671 Wh	proxy	proxy	MODELED
Generic conventional frontier query [S4]	0.34 Wh (IQR 0.18–0.67)			MODELED
Generic test-time-scaling query [S4]	4.32 Wh (IQR 2.38–7.38)			MODELED

The Gemini figure's boundary: accelerators 0.14 Wh (58%), host CPU+DRAM 0.06 (25%), idle provisioned machines 0.02 (10%), data-center overhead 0.02 (8%). An accelerator-only boundary gives 0.10 Wh — 2.4× smaller — which is why boundaries are never dropped when comparing figures.

7.3 Model priors (ordinary short interaction: ~100 in / ~300 out, no tools)

Model	Low–high Wh	Centre	Basis
Gemini 3.6 Flash	0.15–0.40	0.24	Gemini Apps median adapted to the efficient tier
Gemini 3.1 Pro	0.30–1.00	0.50	Prior above the product median
GPT-5.6 Luna	0.20–0.60	0.34	ChatGPT reported average + frontier baseline
GPT-5.6 Terra	0.30–0.90	0.50	Between efficient and flagship

Model	Low–high Wh	Centre	Basis
GPT-5.6 Sol	0.40–1.20	0.67	GPT-5 short/minimal routing proxy
Claude Sonnet 5	0.60–1.20	0.90	Claude 3.7 proxy, widened
Claude Opus 5	0.80–2.00	1.20	Prior; thinking on by default
Claude Fable 5	1.00–3.00	2.00	Prior; adaptive thinking always on
Claude Mythos 5	1.00–3.00	2.00	Own profile mirroring Fable; never aliased to it

7.4 High-reasoning / agentic priors

Model	Modes	Medium band (Wh)	Long band (Wh, open-ended)
Gemini 3.1 Deep Think	deep-think	2.0–7.0	5.0–25.0
Gemini 3.1 Pro	high, xhigh, max	1.5–7.0	4.0–20.0
GPT-5.6 Sol	high, xhigh, max, pro	3.0–18.0	10.0–35.0
Claude Opus 5	xhigh, max	3.0–20.0	8.0–35.0
Claude Fable 5	high, xhigh, max	4.0–25.0	10.0–40.0

Centres are **geometric means** √(low·high): for a band spanning an order of magnitude the arithmetic midpoint sits far above the bulk of the distribution. Anchors: the 4.32 Wh test-time-scaling median and the GPT-5 routing proxy (0.67 / 2.33 / 17.15 / 33.8 Wh).

7.5 Assembly algorithm, step by step

```
1. expected output: observed tokens if known; else median for the regime
      (ordinary 300, reasoning 5000) with band ×0.4 … ×2.5
2. scenario: short if in ≤ 400 and out ≤ 600; medium if in ≤ 4000 and out ≤ 2500; else long
3. regime: 'reasoning' if reasoning ∈ {high, xhigh, max, pro, deep-think}
           or any agentic tool (deep-research, research, computer-use, agent,
           multi-agent, codex, code-execution); else 'ordinary'
4. model = effectiveModel || selectedModel || null      (Auto with nothing exposed stays null)
5. base band:
     reasoning + model prior covering this mode → medium or long band
     reasoning, no prior   → generic 2.38–4.32–7.38 Wh; long → widen high to 17.15 Wh proxy
     ordinary + model prior → short prior × scenario multiplier (medium 2.5–3.5×, long 5–7×)
     ordinary, unknown model → generic 0.18–0.34–0.67 Wh × scenario multiplier
6. token fit (OpenAI/unknown: GPT-4o-Azure; Anthropic: Claude-3.7-AWS; Google: none):
     E ≈ fixed + a·min(in,10000) + b·min(out,1500)
     may only RAISE the band's high (widenUpTo) — never lower the evidence-backed floor
     beyond the fit range: high × min(3, 1 + 0.5·(excess − 1))   (sublinear widening)
7. overhead tools (web-search, browsing, image, video, audio, canvas, artifacts, file-search,
   connected-apps): mul = min(2, 1 + 0.15·n); central × (1+mul)/2; high × mul
8. calls observed > 1 → multiply band by call count
9. lowerBound = agentic tools OR routing auto OR calls > 1
10. confidence = profile confidence, −1 step if no model, −1 more if Auto without effective model
11. carbon, water, product anchor, disclosures (below)
```

Token fit	fixed (Wh)	per input token	per output token	Fitted to
gpt_4o_azure	0.121	0.000131	0.000963	GPT-4o on Azure, 3 published scenarios
claude_3_7_aws	0.118	0.000147	0.002724	Claude 3.7 Sonnet on AWS

These are PromptFootprint's own least-squares fits, not equations the authors published, and describe 2025 models — which is why they shape the band only upward.

7.6 Carbon

Factor	gCO2e per Wh	Accounting	Applied to
google_fleet	0.125–0.1325–0.14	market-based + allocated embodied	Google
azure_openai_proxy	0.35	location-based	OpenAI
aws_anthropic_proxy	0.287	location-based	Anthropic reference line
unknown_grid	0.10–0.245–0.60	unspecified	Anthropic main range (multi-cloud) and unknown providers

`combineCarbon()` refuses to merge market-based and location-based figures — doing so would rank providers by an artefact of their reporting choice.

7.7 Water

Factor	mL per Wh	Boundary	Field
google_product_cooling	1.15 (anchor 0.24 Wh → 0.26 mL)	cooling + associated infrastructure	water.cooling (Google)
chatgpt_reported	0.322 / 0.34 ≈ 0.947	undisclosed	water.reported (OpenAI)
azure_full_operational	4.35 + 0.30/1.12 ≈ 4.618	on-site cooling + electricity generation	water.fullOperational (OpenAI)
aws_full_operational	5.11 + 0.18/1.14 ≈ 5.268	on-site cooling + electricity generation	water.fullOperational (Anthropic)
hyperscaler range	4.618–5.268	operational	unknown providers

Cooling water and full-operational water are different quantities in different fields; `combineWater()` refuses to merge boundaries and the UI always shows the caveat. Operational proxies exclude hardware-manufacturing water.

7.8 Worked examples (actual estimator output)

Produced by running `PFEstimator.estimate()` under Node on this repository:

Input	Scenario / regime	Energy Wh (low–central–high)	CO2e g (central)	Water mL (central, field)	Evidence / confidence
Claude Sonnet 5, 100 in / 300 out	short / ordinary	0.60 – 0.90 – 1.20	0.22	4.74 full-operational	ENGINEERING_PRIOR / low
Claude Fable 5, high effort, 1k / 1k	medium / reasoning	4.0 – 10.0 – 25.0	2.45	52.7 full-operational	ENGINEERING_PRIOR / low
GPT-5.6 Sol, no reasoning, 100 / 300	short / ordinary	0.40 – 0.67 – 1.20	0.23	3.09 full-operational	ENGINEERING_PRIOR / low
ChatGPT Auto (no model exposed), 100 / 300	short / ordinary	0.18 – 0.34 – 0.67	0.12	1.57 full-operational	MODELED / very-low; lower bound
Gemini 3.6 Flash, 100 / 300	short / ordinary	0.15 – 0.24 – 0.40	0.03	0.26 cooling	ENGINEERING_PRIOR / low
Unknown provider, 20k / 2k	long / ordinary	0.90 – 2.04 – 5.39	0.50	10.1 full-operational	MODELED / low; fit clamped

7.9 Display precision

Precision is capped by evidence: ≥10 → integer, ≥1 → 1 decimal, ≥0.01 → 2 decimals, else 3; both ends of a range share one precision chosen from the larger endpoint ("0.8–2.0 Wh", never "0.80–2.0").

7.10 Context carried with every estimate

Each result includes the **product anchor** for the surface (e.g. Gemini 0.24 Wh) flagged `modelSpecific: false` with a warning; the list of assumptions in plain language; source ids and the newest source date; `modelSnapshotId` and `profileId`; the token fit used; and disclosures generated from the evidence class, lower-bound flag and phase.

8. Prompt Savings Math

Shortening a prompt removes part of **prefill**, which is a minority of interaction energy at short and medium lengths. Output decoding, hidden reasoning, routing, tools and fixed serving cost do not shrink.

Scenario	Input share of interaction energy (low–high)	Central (geometric mean)
Short	1% – 5%	≈2.24%
Medium	5% – 15%	≈8.66%
Long	25% – 50%	≈35.4%

```
fraction = tokensAvoided / originalInputTokens
gross    = share × fraction                                  (per low/central/high)
penalty  = expectedOutputGrowthFraction × (1 − share)
net      = gross − penalty                                   (may be ≤ 0)
energySaved = estimate.energyWh × net ; carbon/water follow the same fraction and keep their labels
```

Example (actual run): halving a 200-token Sonnet 5 prompt (short scenario) yields a projected total reduction of **0.5% – 1.1% – 2.5%** of the interaction — not "50% saved". Three distinct numbers are kept apart: `inputReductionFraction` (tokens only), `inputProcessingPct` (gross share), and `totalReductionPct` (net). Before send the wording is "potential input tokens avoided" / "estimated input-processing reduction"; "saved" is reserved for comparing two complete, same-model, same-mode scenarios. A 2026 study of 28,421 trials [S21] found compression effects are provider-dependent and can include output expansion, which is why the penalty term exists.

9. Legacy Model, Heatwave Overlay and Equivalents

9.1 Legacy per-token model (storage schema 1)

```
Annual (OpenAI 2025 disclosure, GPT-4o):  390e9 Wh · 1.3e12 mL · 138e9 g CO2
Tokens/interaction = round(1.3 × (41 + 269)) = 403 ; 2.5e9 messages/day
Annual tokens ≈ 2.5e9 × 403 × 365 ≈ 3.677e14
Energy/token ≈ 1.06e-3 Wh · Water/token ≈ 3.54e-3 mL · CO2/token ≈ 3.75e-4 g
Claude profile = GPT-4o × CLAUDE_RELATIVE_INTENSITY (1.15)
timeFactor = clamp(baselineTokPerSec / observedTokPerSec, 1, 3)   (skip if response < 0.5 s;
             baseline 55 tok/s ChatGPT, 45 tok/s Claude)
```

Records written under this model are stamped `estimatorVersion: "legacy-token-linear-v1"` on migration and left exactly as recorded — history is what the user was shown, and recomputing it would falsify the record. Old flat reasoning multipliers (1.0 / 1.9 / 6.0 / 14.0) were removed in favour of evidence-classed ranges. `calculateQueryImpact()` still runs first in `processQuery`; its numbers are then replaced by the evidence-aware estimate.

9.2 Heatwave / cooling overlay

```
peakPUE(T) = 1.1 + (1.4 − 1.1) × clamp((T − 25) / (40 − 25), 0, 1)
factor     = peakPUE(T) / 1.1                 // 1.0 at ≤ 25 °C … ≈1.27 at ≥ 40 °C
isHot      = feels-like ≥ 32 °C and factor > 1.001
```

Display-only and never stored. The service worker (not the content script, which is blocked by the chat page CSP) finds the nearest of ~20 cloud regions (Haversine distance; e.g. US East N. Virginia, Europe Ireland, Asia Pacific Tokyo), fetches Open-Meteo apparent temperature with coarsened coordinates, caches it, and returns the factor. It never claims to know which data center served a request.

9.3 Real-world equivalents

For intuition only: water → drops/glasses; energy → phone screen-on time (~3 W, so 1 Wh = 1200 s); CO_2 → metres driven (~200 g/km).

10. Token Cutter — Local Prompt Optimizer

One TypeScript engine (`stats-site/src/lib/tokenCutter/`) serves both the dashboard page and the in-page assistant. It is compiled with `esbuild --format=iife --global-name=PFTokenCutter` into `extension/lib/tokenCutter.bundle.js` (committed; `npm run build:cutter`). It works with no API key, no account and no network. Stages are pure functions; nothing mutates the user's text.

#	Stage	What it does
1	protect (protect.ts)	Locks code fences/inline code, balanced-brace JSON, URLs, emails, placeholders ({{x}}, ${x}, <x>, %s), math, quotes, Markdown links, paths, dates, numbers with units, never-remove terms. Byte mask → linear time. Hard vs soft protection.
2	segment (segment.ts)	Blocks then sentences (abbreviation-aware); roles: role, task, context, constraint, example, format, question, meta
3	extract (entities.ts, constraints.ts)	Names, numbers, dates, URLs, emails, file types, technologies, proper nouns, quotes, negations, length limits, must/must-not; constraint keys for length, tone, format, audience, deadline, inclusion, exclusion, language
4	detect (grammar.ts, lexicon.ts, redundancy.ts)	Typos, a/an, doubled words; politeness, filler, hedges, wordy constructions, instruction wrappers, meta-commentary; repeated sentences (both directions), clauses, restated constraints, parallel imperatives
5	generate (suggestions.ts)	Vetoes, de-overlap (highest confidence wins), level gating, scoring
6	apply (apply.ts)	Right-to-left splice; seam repair (orphan punctuation, spacing, re-capitalize); whitespace tidy outside fenced code
7	validate (validate.ts)	Re-extract entities/constraints from the result and compare with the original; critical/warning; culpable-edit repair
8	refine	Re-run 1–7 up to 3 more times; each round must reduce tokens and is validated against the ORIGINAL
9	assess (concision.ts)	"Already concise" only if all 7 conditions hold

10.1 Vetoes

- Touches protected content.
- Reduces the number of non-idiomatic negations ("whether or not" is idiomatic; every other "not" is counted).
- Damages a constraint, unless the edit is a deduplication of that constraint.
- Removes a filler word its context makes meaningful ("just the code", "not really", "very concise").

10.2 Confidence and levels

Score	Bucket	Behaviour
≥ 0.85	High	Accepted by default
0.62–0.85	Medium	Offered, not applied
< 0.62	Low	Offered, never applied automatically

Level	Intent	Auto-applies
Light	Grammar, spelling, obvious filler; keeps voice	"safe" edits ≥ 0.85
Balanced	Cut substantially, keep tone/context/requirements	anything ≥ 0.70
Maximum	Smallest prompt carrying every essential instruction	anything ≥ 0.55

Aggressive floors are safe because everything applied is re-validated against the original and any edit that lost information is rolled back individually via `culpableSuggestionIds()`.

10.3 "Already concise" — seven conditions

- No semantic repetition outstanding
- No filler, politeness, hedge or wordy construction outstanding
- No two instructions that could safely be combined
- No spare formatting whitespace
- No verbose instruction wrapper or empty framing
- Available reduction negligible (< 2 tokens or < 1.5%)
- Another full pass produces nothing

10.4 Memory

On-device preferences (chrome.storage.local / localStorage): tone, length, formatting, style, terminology, names, never-remove terms, standing instructions. Nothing is remembered without an explicit action; only relevant entries apply (trigger words; at most six per prompt); the current prompt always wins over stored preferences; every application is reported. Entries can be weighted 1–5, disabled, exported, imported and deleted.

10.5 Optional Gemini enhanced pass

The client sends a structured instruction (prompt, constraints, verbatim-protected strings, entities) and requires a JSON envelope: `{optimized, preservedConstraints, removedRedundancies, uncertainChanges, protectedContent, meaningScore}`. The response is untrusted: parsed out of fences, shape-checked and sanitized; every protected string must come back byte-identical; then it runs through the same local validator. Any failure (not configured, too long, timeout, network, 429, non-JSON, malformed, protected content changed, information lost) keeps the local result with a plain-language reason.

10.6 Quality evaluation

`npm run eval` in stats-site runs 22 cases; a case that saves more tokens but loses a requirement fails. Documented results: 22/22 passed, 24.3% average reduction (several cases should barely change; verbose real-world prompts reduce 35–60%), 100% constraint and entity preservation, 0% unsafe auto-applied, meaning score 1.000. The extension's `tokenCutterAggression.test.js` runs against the shipped bundle with reduction floors, so a stale bundle fails CI rather than a user's browser.

11. In-Page Writing Assistant

`overlay/assistant.js` is the floating indicator anchored to the composer corner. It is a surface for the Token Cutter engine, not a second optimizer. Three rules: the prompt is the user's (nothing rewritten without an explicit click; every replacement undoable to the exact original string); it must cost nothing to have open (analysis on a typing pause, stale results discarded, mutation bursts coalesced to one re-detect per 250 ms, a 500 ms anchor tick for layout changes no observer reports); failure is quiet.

- Analysis after a 600 ms pause; a saving is offered only if ≥ 2 tokens **and** ≥ 1.5%; otherwise the concision engine says "Already concise" or "Little left to cut", with reasons.
- "Meaning preserved" is shown only when `validated && ok`.
- Savings are shown as two ranges from `projectInputSavings`: input-processing share and total interaction share.
- Model changes arrive by subscription from the single detector; the assistant re-labels its pill, cancels in-flight work and re-optimizes — it never scrapes the page for a model itself.
- Enhanced (remote) mode caps at 4,000 characters, matching the Worker.

11.1 Offline spell / writing checker

`spellChecker.js` detects misspellings (curated typo map + Typo.js with a bundled Hunspell en_US dictionary), capitalization (sentence starts, "I"), punctuation (double spaces, space before punctuation, repeats, missing terminal punctuation) and basic grammar (a/an, repeated words). Suggestions are `{type, original, suggestion, reason, safe}`; only safe ones join "Accept all safe fixes". British spellings are not "corrected".

12. AI Proxy and Service-Worker Rate Control

12.1 Cloudflare Worker (proxy/worker.js)

Mode	Used by	Returns	Input cap
shorten (default)	extension optimizer	{ rewritten }	4,000 chars
improve	writing assistant	{ improved, rewritten }	4,000 chars
cutter	dashboard Token Cutter	{ cutter } JSON envelope	12,000 chars

Model gemini-2.0-flash. The API key exists only as a Cloudflare secret (`wrangler secret put GEMINI_API_KEY`); clients only know the public URL. Abuse protection: method/shape validation, input and output caps, and an in-memory per-IP limit of 30 requests per minute (best effort; KV/Durable Objects suggested for stronger guarantees).

12.2 Service-worker client

Mechanism	Setting
Provider resolution	Configured Worker URL first; advanced user-supplied Gemini key second; otherwise local-only
Token bucket	capacity 5, refill 10 per minute (cross-tab, the worker is a singleton)
TTL cache	100 entries, 10 minutes; in-flight de-duplication of identical requests
Timeout	10 s per request via AbortController
Retries	Up to 2 with backoff + jitter for 5xx/network only; never for 429
429 cooldown	Global; honours Retry-After, default 30 s, clamped at 120 s
Health stats	pf_ai_stats: success rate over real network attempts (not-configured/cached/throttled excluded)
Failure contract	Always returns text '' on failure so the UI falls back to local suggestions

13. Local Storage and Data Model

All persistence is `chrome.storage.local` via `lib/storage.js` (loaded in content scripts, the service worker and Node tests). Each tab owns one session id and each session is its own key, so concurrent tab writes never clobber a shared array.

Key	Contents
pf_userId	Anonymous UUID
pf_config	overlayEnabled, energyPerTokenMultiplier, writingChecksEnabled, debug, assistant level (light/balanced/maximum) and mode (local/enhanced), heatwave location, Worker URL, optional Gemini key
pf_session_<id>	Session object with totals and queries nested inline
pf_savings	Realized savings totals and a per-day map
pf_schema	Storage schema version (currently 2)
pf_observed_models	Discovery registry of unknown model labels (≤ 50)
pf_ai_stats	AI layer health metrics
pf_auth	Supabase session (service worker only, via custom storage adapter)
chrome.storage.session session_<tabId>	Tab → session mapping for closing sessions on tab removal

13.1 Per-query record

```
platform, promptTokens, responseTokens, totalTokens, responseTimeMs
energyWh, waterMl, co2G                      (central values; × user multiplier)
energyWhLow, energyWhHigh
waterCoolingMl, waterFullOperationalMl, waterBoundary
carbonScope, carbonAccounting
evidence, confidence, lowerBound
selectedModel, canonicalModel, effectiveModel, routing, reasoningMode, tools
modelSnapshotId, estimatorVersion              (no prompt or response text, ever)
```

Aggregation helpers (`computeTotals, computeWeeklyStats, computePlatformBreakdown`) are pure and unit-tested. Migration to schema 2 is additive: legacy records are stamped, not recomputed.

14. Optional Accounts and Cross-Device Sync

Supabase (Auth + Postgres + Row-Level Security) with email/password. Signed-out use is unchanged. The service worker owns the client and session; the dashboard drives auth by messages (AUTH_SIGNUP, AUTH_LOGIN, AUTH_LOGOUT, AUTH_STATUS, AUTH_SET_NAME, AUTH_DELETE, SYNC_NOW) and never sees tokens.

14.1 Schema (supabase/migrations/0001_init.sql)

Table	Key	Columns
profiles	user_id	anon_client_id, display name (0002), timestamps
user_settings	user_id	overlay_enabled, writing_checks_enabled, energy_per_token_multiplier ($0 < x \leq 20$)
session_stats	(user_id, session_id)	platform ∈ {chatgpt, claude, unknown}, start/end, totals for tokens, energy, water, CO2, response time, query_count (all ≥ 0)
savings_daily	(user_id, day)	count, tokens, energy_wh, water_ml, co2_g

- RLS on every table: select/insert/update/delete only where `auth.uid() = user_id`, so the shipped anon key is safe.
- Trigger `handle_new_user` creates profile and settings rows; `delete_user()` (security definer, authenticated only) deletes the caller's auth user, cascading all rows.
- Upload payload is built by a **whitelist** (`syncPayload.js`): numbers and non-sensitive settings only. Prompt/response text, the Gemini key, the Worker URL, positions and the debug flag are excluded by construction.
- Idempotent: upserts on stable keys; savings totals are recomputed from the daily map, never accumulated (`syncMerge.js`). First login runs a one-time "claim" guarded by `pf_auth.claimedFor`.
- Triggers: dashboard open, "Sync now", hourly `pf_sync` alarm. Failures return quietly and never wipe local data.

15. Dashboard, Popup and Public Site

`stats-site/` is a React 19 + Vite + Tailwind 4 app built into `extension/dashboard/` (the extension's options page) and also deployable as a public site with marketing pages.

View	Purpose
Sessions (SessionList)	Per-session totals and queries
Weekly stats	Trends with Recharts
Savings	Realized savings totals and daily history
Awards	Gamified milestones
Weather card	Heatwave context for the chosen region
Token Cutter	Full optimizer UI: diff, per-suggestion accept/reject, impact panel, memory panel, keyboard shortcuts
Guide	Methodology explanation
Settings / Account	Preferences, Worker URL, sign-in and sync

The toolbar popup (`extension/popup/`) shows current model state, the draft estimate, savings and AI health, using the same `present.js` formatting as the in-page panel so the two cannot disagree.

16. Security and Privacy Model

- **No prompt or response text is stored or transmitted** by tracking. Snapshots keep an FNV-1a hash and token count. Text leaves the device only when the user explicitly invokes an AI rewrite, and then only to the configured Worker.
- **No network interception and no page-world injection.** Detection reads accessible DOM state only.
- **Sender validation** in the service worker: message sender id must equal the extension id, and tab senders must come from an extension page or an allowed origin.
- **Secrets:** the Gemini key lives only in Cloudflare; Supabase tokens only in the service worker's storage area.
- **Strict extension CSP** and a minimal host-permission list.
- **Least-privilege database** via RLS; account deletion cascades.
- Supporting documents: docs/PRIVACY.md, SECURITY.md, DATA_DELETION.md, TERMS.md.

17. Testing and Quality

Suite	Command	Covers
Extension unit tests (33 files)	cd extension && npm test (node:test + jsdom)	model detection, reasoning detection, model presentation, estimator, environmental model, heatwave, token estimator, prompt size, composer, platforms, storage, savings, sync payload/merge, auth state/service, AI client, proxy config, spell checker, writing lexicon/format, assistant + state, worker, timezone, token-cutter aggression on the shipped bundle
End-to-end	npm run test:e2e (extension/e2e/run.js)	Driving the extension against provider fixtures
Engine tests	cd stats-site && npm test	engine, pipeline, parity with the extension's token/impact math
Engine evaluation	npm run eval	22-case quality benchmark (Section 10)
Database	supabase test db	RLS policies (supabase/tests/rls.test.sql)
Full check	npm run check (stats-site)	typecheck + lint + test + build

Notable performance test: an idle page must cost zero extra `querySelectorAll` calls. docs/TESTING.md contains a manual checklist for the ChatGPT lifecycle (new chat + thinking mode), including the expected debug-log sequence.

18. Observations From This Review

Points noticed while tracing the code for this document. They are observations, not changes — nothing in the repository was modified.

#	Observation	Where	Effect
1	Gemini is a supported content-script host, but the service worker's ALLOWED_ORIGINS lists only chatgpt.com, chat.openai.com and claude.ai.	extension/background.js (ALLOWED_ORIGINS, isValidSender)	Messages from Gemini tabs (REGISTER_SESSION, OPTIMIZE_PROMPT, GET_WEATHER_ADJ, …) are rejected as "Unauthorized sender", so session-tab mapping and AI rewrites likely fail on Gemini.
2	Supabase session_stats.platform allows only chatgpt / claude / unknown, and syncPayload maps everything else to unknown.	supabase/migrations/0001_init.sql; extension/lib/syncPayload.js	Gemini sessions sync without their platform label.
3	The AI cache key is built with a literal NUL byte as separator (mode + "\0" + text) written raw into the source.	extension/background.js:383	Works, but makes tools such as grep treat the file as binary; an escaped "\u0000" would be clearer.
4	Model catalog is dated 2026-08-06; any newer model shows its exact label but gets a provider-level estimate.	extension/lib/models/catalog.js	By design; pf_observed_models is the signal to update it.
5	Legacy server/ is unused but still present.	server/	Reference only; not deployed.

19. Appendix A — File Map

Path	What lives there
manifest.json	MV3 manifest: permissions, CSP, content-script order, popup, options page
extension/content.js	Orchestrator in the page: sessions, submit hook, polling capture, detection wiring, pill and modal
extension/background.js	Service worker: sender validation, sessions, AI proxy client, weather, auth, sync
extension/lib/models/	Model identification (Section 4)
extension/lib/estimator.js, env/	Evidence-aware estimator; sources, profiles, factors, required copy
extension/lib/platforms.js, composer.js	Per-site message DOM; composer discovery and text reading
extension/lib/tokenEstimator.js, promptSize.js	Token approximation; personal prompt-size nudge
extension/lib/environmentalModel.js, constants.js	Legacy per-token model, response-time factor, heatwave PUE model
extension/lib/weatherService.js	Cloud-region table, Haversine, Open-Meteo fetch/geocode
extension/lib/storage.js	Local persistence, aggregation, migration
extension/lib/sync*.js, auth*.js, supabaseClient.js	Accounts and sync
extension/lib/aiClient.js, proxyConfig.js	Rate-limit primitives, status classification, Worker URL config
extension/lib/spellChecker.js, writingLexicon.js, writingFormat.js, dict/, vendor/typo.js	Offline writing checks
extension/lib/tokenCutter.bundle.js	Compiled Token Cutter engine (generated)
extension/overlay/	In-page assistant and overlay CSS
extension/popup/, extension/dashboard/	Toolbar popup; built dashboard
stats-site/src/lib/tokenCutter/	Token Cutter TypeScript source
proxy/	Cloudflare Worker
supabase/	Migrations, config, RLS tests
docs/	MODEL_DETECTION, TOKEN_CUTTER, ACCOUNTS, PRIVACY, SECURITY, TESTING, deployment, store listing
METHODOLOGY.md	Environmental methodology and sources

20. Appendix B — Sources Referenced by the Code

Id	Source
S1	Google — Measuring the Environmental Impact of Delivering AI at Google Scale
S2	Sam Altman — The Gentle Singularity
S3	OpenAI Academy — Environmental Impact of AI
S4	Oviedo et al. — Energy use of AI inference, efficiency pathways, and test-time scaling, Joule (2026)
S5	Jegham et al. — How Hungry is AI? v6
S6	Anthropic Transparency Hub
S7–S12	Anthropic model, thinking and effort documentation
S13–S16	OpenAI API model docs and ChatGPT model-availability help
S17–S20	Google DeepMind model cards and Gemini Apps help
S21	Johnson — The Compression Paradox in LLM Inference (2026 preprint)

Full titles, URLs and dates are in `extension/lib/env/sources.js`, the single place any displayed figure resolves its citation from. Legacy constants also cite Parasharami, "A Token-Level Framework for Quantifying ChatGPT's Environmental Impacts" (Vanderbilt YSJ), and the OpenAI 2025 sustainability disclosure.